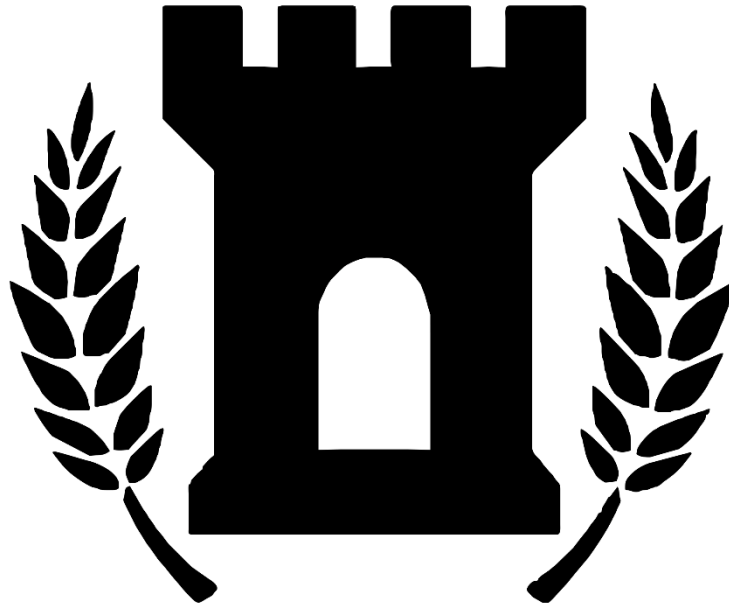


## Feldfeste Operation Instruction



**Feldfeste Games Studio**

### **FF-OI-17: "Attribute Components"**

#### **Document History**

| Revision | Content                   | Editor | Time       |
|----------|---------------------------|--------|------------|
| 01       | Initial Document Creation | Arina  | 10.09.2026 |
|          |                           |        |            |
|          |                           |        |            |



## Table of Contents

|                               |    |
|-------------------------------|----|
| Overview.....                 | 3  |
| Base Attribute Component..... | 4  |
| Functions.....                | 4  |
| Variables.....                | 5  |
| Event Dispatchers.....        | 5  |
| Adding a new attribute.....   | 5  |
| Health Component.....         | 6  |
| Functions.....                | 6  |
| Variables.....                | 6  |
| Events.....                   | 7  |
| Stability Component.....      | 8  |
| Functions.....                | 8  |
| Variables.....                | 8  |
| Events.....                   | 8  |
| Experience Component.....     | 9  |
| Adding experience.....        | 9  |
| New level formula.....        | 10 |
| Functions.....                | 10 |
| Variables.....                | 11 |
| Events.....                   | 11 |
| Morale Component.....         | 12 |
| Changing morale.....          | 12 |
| Design note.....              | 12 |
| Functions.....                | 12 |
| Variables.....                | 12 |
| Events.....                   | 13 |
| Stamina Component.....        | 14 |
| Functions.....                | 14 |
| Variables.....                | 14 |
| Events.....                   | 14 |



## Overview

This document describes the attribute system architecture. The system is built around a single reusable base component that encapsulates one float value — designed so that adding a new attribute (health, stamina, experience, etc.) requires no duplication of core logic.

This document covers **BaseAttributeComponent** and its child components.

### Key Concepts

- **BaseAttributeComponent** — parent class storing and managing a single float attribute; provides init, value access, and optional regeneration
- **HealthComponent** — health, with restore and damage handling; dies at 0
- **StabilityComponent** — health for buildings; can be restored at 0 HP, cannot evade or block damage
- **ExperienceComponent** — experience and level progression, with optional manual level-up
- **MoraleComponent** — morale that can influence actor behaviour; regenerates toward a configurable baseline instead of max
- **StaminaComponent** — stamina spent on actions and restored by resting

As a general rule, attribute components are only responsible for storing and managing their own value — not for deciding why it should change or what triggers that change. That decision belongs to other systems that call into them.



## Base Attribute Component

Base class located:

`/CA_Code/Blueprints/ActorsComponents/AttributeComponents/BaseAttributeComponent.uasset`

Base component that stores and manages a **single float value** with encapsulated access. Designed to be reused for any float-based attribute on an actor — health, stamina, experience, etc.

**BaseAttributeComponent** is an abstract class — it cannot be added directly to an actor and only exists to be extended by child components.

### Functions

Grouped into three categories: **Attribute Default**, **Attribute**, and **Regeneration**.

- **Attribute Default** — handles initialization: setting the starting max value, running init logic, and checking whether initialization has completed.
- **Attribute** — core read/write logic for the value: setting, updating, and clamping it, plus getters for current value, max value, percent, and full/zero state.
- **Regeneration** — controls automatic value recovery over time: starting, stopping, and ticking regeneration, checking whether it can run, and configuring its rate and delay.

Most functions are **protected** — intended to be called by this component and its child classes. Only functions meant to be used externally (e.g. by other systems querying the attribute) are **public**.

**⚠** When **overriding these functions** in a child class, make sure to keep **calling any relevant events** that were present in the base implementation — skipping them can silently break dependent systems (UI, other components) that rely on those broadcasts.

### Value Setters

The component exposes three layered functions for changing the attribute's value, each building on the one below it.

- **UpdateValue** (*protected*) — the base function for changing the attribute by a delta. Takes a **Delta** value, clears and restarts the auto-regeneration timer (if regeneration is enabled), then calls **SetValue** internally.
  - Restarting the timer ensures regeneration doesn't resume mid-cycle after an interruption — e.g. if the character takes damage, the timer resets and the initial regeneration delay starts over, rather than continuing as if nothing happened.
- **SetValue** (*protected*) — sets the attribute to an exact new value. Calls **SetValueSilent** internally, then broadcasts **OnAttributeValueChanged**.
  - Used directly by the regeneration tick (instead of UpdateValue), since UpdateValue would reset the regeneration timer on every tick.



## FF-OI-17: "Attribute Components" Rev(01)

- **SetValueSilent** (*protected*) — the final setter. Clamps the new value between 0 and MaxValue, then assigns it to AttributeValue. Does not broadcast any events.

### Variables

#### Attribute Default

- **InitMaxValue** (*public, Float*) — initial max value used only once, during initialization. Exposed as public because Blueprints have no **protected** access modifier — this is a workaround to allow child classes to set the starting max value without exposing the runtime **MaxValue** itself, which must stay encapsulated.
- **MaxValue** (*private, Float*) — current max value of the attribute, used in all runtime calculations. Kept private to preserve encapsulation; use the corresponding getter/setter instead of accessing it directly.
- **InitCompleted** (*private, Boolean*) — tracks whether initialization has finished.

#### Attribute

- **AttributeValue** (*private, Float*) — current value of the attribute.

#### Regeneration

- **bCanAutoRegenerate** (*private, Boolean*) — whether automatic regeneration is enabled for this attribute.
- **RegenerationModifier** (*private, Float*) — amount added per regeneration tick.
- **RegenerationUpdateTime** (*private, Float*) — interval between regeneration ticks.
- **RegenerationInitDelay** (*private, Float*) — delay before regeneration starts.
- **AutoRegenerationTimer** (*private, Timer Handle*) — handle for the active regeneration timer.

### Event Dispatchers

- **OnAttributeValueChanged** — broadcast whenever the attribute's value changes.
  - **Value** — the new current value.
  - **DeltaValue** — the amount the value changed by.
  - **MaxValue** — the current max value, provided for convenience (e.g. for UI updates).
- **OnMaxValueUpdated** — broadcast whenever the **max value changes**.
- **OnInitComplete** — broadcast once initialization has finished.

### Adding a new attribute

Create a child class of **BaseAttributeComponent**. Examples: **StaminaComponent**, **HealthComponent**, **ExperienceComponent**



## Health Component

Located:

`/CA_Code/Blueprints/ActorsComponents/AttributeComponents/HealthComponent.uasset`

Child of `BaseAttributeComponent`. Stores the **health value** of an actor — health ranges from 0 to the max value.

When health reaches 0, the actor is **considered dead**: auto-regeneration stops and health can no longer be restored externally.

## Functions

Grouped into Regeneration, Health, and Damage.

- **Regeneration** — overrides `ShouldStartRegeneration` to prevent regeneration from starting on a dead actor.
- **Health** — handles restoring and querying health state:
  - `IsAlive` (public) — whether the actor is currently alive.
  - `TryToRestoreHealth` (public) — attempts to restore a given amount of health (like healing for living entities). Takes the amount to add as `Health Point`. Internally checks `IsRestoreBlocked`, then applies the change via `SetValue` (not `UpdateValue`) — since restoring health from an external source is a one-off heal and should not reset the regeneration timer.
  - `IsRestoreBlocked` (protected) — blocks restoration if the incoming value is less than or equal to 0, if the actor is already at full health, or if the actor is dead.
  - `Killed` (private) — internal handling for when health reaches 0.
- **Damage** — handles incoming damage.

**HealthComponent** subscribes to the owner actor's standard `OnTakeDamage` event — this is the main damage pipeline.

A separate event, `ApplyEffectDamage`, handles damage coming from status effects (e.g. Burn) rather than direct attacks. It exists as a distinct path so that applying effect damage doesn't reapply the effect itself, and because effect damage cannot be evaded or blocked. Effect damage does not trigger `UnderAttack`.

*For full details on damage calculation, resistances, piercing, and status effects, see the "Damage system" instruction.*

## Variables

No new variables are introduced — `HealthComponent` relies entirely on the variables inherited from `BaseAttributeComponent`.



## FF-OI-17: "Attribute Components" Rev(01)

### Events

- **OnZeroHealth** — broadcast when health reaches 0.
- **OnReceiveDamage** — broadcast when the actor takes damage.
- **OnBlockedDamage** — broadcast when incoming damage is fully blocked.
- **OnEvadeDamage** — broadcast when incoming damage is evaded.
- **UnderAttack**— broadcast when the actor is attacked.



## Stability Component

Located:

`CA_Code/Blueprints/ActorsComponents/AttributeComponents/StabilityComponent.uasset`

Health component for buildings. Child of `HealthComponent`, extending its behaviour with logic specific to structures.

Introduces a new variable, `ConstructionComplete` — set to `true` once, the first time the building finishes construction. It stays `true` from that point on, even if the building's HP later drops below max, to keep the building's mesh displaying correctly rather than reverting to an "unfinished" state.

Buildings differ from regular actors in two ways: they can be restored even at 0 HP, and they cannot evade or block incoming damage — `IsRestoreBlocked` and `TryEscapeDamage` are overridden accordingly. A building can also receive an HP bonus or penalty depending on the terrain it's built on.

## Functions

Grouped into Attribute, Health, and Damage.

- **Attribute** — `SetValue` is overridden to keep `ConstructionComplete` in sync once the building reaches full HP for the first time.
- **Health** — `IsRestoreBlocked` is overridden to allow restoration even at 0 HP; `HasBeenConstructed` (public) checks whether construction has completed.
- **Damage** — `TryEscapeDamage` is overridden so buildings always take full damage, with no evade or block; `StartBuild` (public) is called at the start of a new building's construction; `CheckTerrainBonusAndPenalty` (public) applies the terrain-based HP bonus/penalty.

## Variables

- **ConstructionComplete** (*private, Boolean*) — whether the building has completed construction at least once.

## Events

- **OnConstructionComplete** — broadcast the first time the building finishes construction.





## Experience Component

Located:

`CA_Code/Blueprints/ActorsComponents/AttributeComponents/ExperienceComponent.uasset`

Child of `BaseAttributeComponent`. One of the more complex attributes — introduces several new concepts:

- The inherited `BaseAttributeComponent` value holds the unit's committed experience — experience toward the current level.
- `Level` (*integer*) — has a current, max, and start value. The minimum possible level is 1.
- `AccumulatedExp` — a separate buffer for experience the unit has earned but not yet committed. Not to be confused with committed experience above — this value only exists to be committed later.

### Adding experience

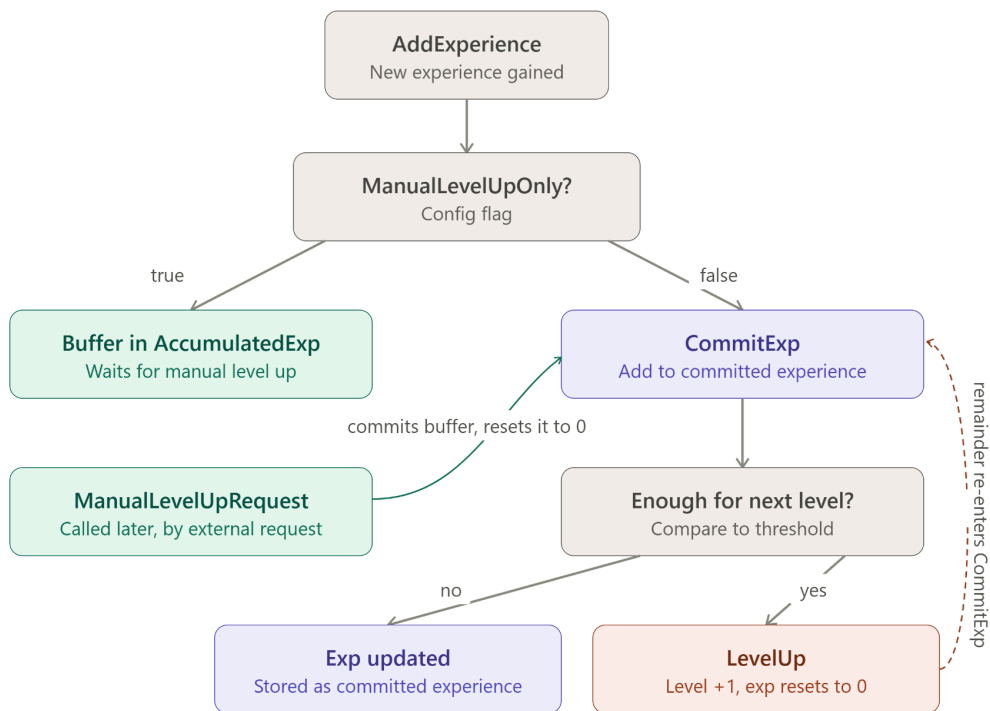
New experience is added through `AddExperience`. Behaviour depends on the `ManualLevelUpOnly` setting:

- If **true**, incoming experience is first buffered into `AccumulatedExp`. It only gets committed — and can trigger a level up — on an external request (`ManualLevelUpRequest`), which commits the buffer via `CommitExp` and resets `AccumulatedExp` to 0.

*This lets gameplay defer level-ups to a moment outside combat: e.g. a unit earns experience mid-fight, but only actually levels up once it returns to camp and rests.*

- If **false**, experience is committed immediately.

Committing experience (`CommitExp`) adds it to the committed experience value. If the total reaches the threshold required for the next level, the unit levels up (`LevelUp`), committed experience resets to 0, and any leftover experience is recommitted — so a large enough gain can trigger multiple level-ups in a row.



**Gray** — entry point and decision checks; **purple** — committed experience (the inherited base attribute value); **teal** — buffered, accumulated experience; **coral** — level change.

## New level formula

The experience required for a given level is configurable via **ExperiencePerLevelMultiplier** and **ExperiencePerLevelConstant**:

$y = kx + b$ , where  $x$  is the level,  $k$  is ExperiencePerLevelMultiplier, and  $b$  is ExperiencePerLevelConstant.

Level 1 is the starting level and requires 0 experience — the formula applies from level 2 onward.

## Functions

Grouped into **Attribute Default**, **Level**, and **Experience**.

- **Attribute Default** — **Init** sets up the component on start.
- **Level** — **SetCurrentLevel**, **LevelUp**, **SetupNextLevelParam** handle level progression internally; **ManualLevelUpRequest** (*public*) commits buffered experience on external request; **GetCurrentLevel** and **IsMaxLevel** (*public*) expose level state.



## FF-OI-17: "Attribute Components" Rev(01)

- **Experience** — `SetExpForNextLevel`, `CanAddExp`, `SetAccumulatedExp`, `CommitExp` handle the internal experience flow; `GetExpRequiredForLevel`, `HasEnoughExpToLevelUp`, `GetExpRequiredForLevelUp`, `GetCurrentExperience`, and `AddExperience` (*public*) expose experience state and the main entry point for adding experience.

### Variables

#### Level

- **CurrentLVL** (*private, Integer*) — current level.

#### Experience

- **AccumulatedExp** (*private, Float*) — buffered, uncommitted experience.
- **ExperienceForNextLevel** (*private, Float*) — cached experience threshold for the current level.

#### Experience And Level Default

- **MaxLVL** (*public, Integer*) — maximum level.
- **StartLVL** (*public, Integer*) — starting level.
- **StartEXP** (*public, Float*) — starting experience.
- **ManualLevelUpOnly** (*public, Boolean*) — see "Adding experience" above.
- **ExperiencePerLevelMultiplier** (*public, Float*) — used in the threshold formula.
- **ExperiencePerLevelConstant** (*public, Float*) — used in the threshold formula.

### Events

- **OnLevelUp** — broadcast when the unit levels up.
- **OnMaxLevel** — broadcast when the unit reaches max level.
- **OnNewExpForLevel** — broadcast when the threshold for the next level is recalculated.
- **OnAccumulatedExpChanged** — broadcast when `AccumulatedExp` changes.



## Morale Component

Located:

`CA_Code/Blueprints/ActorsComponents/AttributeComponents/MoraleComponent.uasset`

Child of `BaseAttributeComponent`. Stores morale, which can influence the behaviour of various actors. Introduces one new concept:

- **Baseline** — a target value that regeneration always pulls morale back to, moving the value up or down as needed. Unlike other attributes, morale doesn't regenerate toward its **max value** — it regenerates toward this **baseline** instead. Most of the component's logic exists to support this behaviour.

### Changing morale

- `ApplyMoraleEvent` — the main way morale changes. Takes a `MoraleEvent` enum; the actual value to apply is looked up from the `DT_CA_MoraleEvents` data table (`CA_Code/DataTables/DT_CA_MoraleEvents.uasset`).
- `UpdateMorale` — a public function to change morale by an arbitrary delta, for other cases not covered by predefined morale events.

### Design note

**⚠** *The Morale Component is **only responsible for storing and regenerating the morale value**. It must never contain logic that reacts to external events or triggers gameplay side effects — that responsibility belongs to `MoraleListenerComponent` (`CA_Code/Blueprints/ActorsComponents/MoraleListenerComponent.uasset`), which listens for requests and decides when and how morale should change.*

### Functions

- **Regeneration** — overrides the base regeneration logic so the value moves toward the baseline each tick, instead of toward max.
- **Baseline** — restoring, updating, and querying the current baseline value.

Most functions are protected — intended to be called by this component and its child classes.

Only functions meant to be used externally are public. `UpdateMorale` and `ApplyMoraleEvent`, described above, are the two public entry points for changing morale.

### Variables

#### Regeneration



## FF-OI-17: "Attribute Components" Rev(01)

- **DefaultRegenModifier** (*public, Float*) — default amount the value moves per regeneration tick.
- **DefaultRegenInitDelay** (*public, Float*) — default delay before regeneration starts.
- **DefaultRegenUpdateTime** (*public, Float*) — default interval between regeneration ticks.

### Morale

- **DefaultMoraleBaseline** (*public, Float*) — the baseline value morale regenerates toward, by default.

### Baseline

- **MoraleBaseline** (*private, Float*) — the current baseline value, which regeneration pulls morale toward.

### Events

- **OnMoraleBaselineUpdated** — broadcast when the baseline value changes.



## Stamina Component

Located:

`CA_Code/Blueprints/ActorsComponents/AttributeComponents/StaminaComponent.uasset`

Child of `BaseAttributeComponent`. Stores stamina, which can affect various actor parameters. Stamina is spent on actions via `ReduceStamina`, called by request from other systems.

Stamina only regenerates while the actor is resting — regeneration is off by default and turns on only between `StartRest` and `StopRest`.

## Functions

Grouped into `Attribute Default`, `Rest`, and `Stamina`.

- **Attribute Default** — handles initialization.
- **Rest:**
  - `StartRest` — begins resting: turns on auto-regeneration and broadcasts `OnStartRest`.
  - `StopRest` — ends resting: turns off auto-regeneration.
  - `IsResting` — checks whether the actor is currently resting.
  - A separate group of functions lets rest parameters be overridden per instance and restored to their default — e.g. resting in different buildings could restore stamina at different speeds.
- **Stamina** — reducing stamina by request, checking whether the actor is exhausted, and reacting internally when stamina changes.

## Variables

Rest

- **DefaultRestInitDelay** (*public, Float*) — default delay before regeneration starts once resting begins.
- **DefaultRestUpdateTime** (*public, Float*) — default interval between regeneration ticks while resting.
- **DefaultRestModifier** (*public, Float*) — default amount restored per regeneration tick while resting.

## Events

- **OnExhaust** — broadcast when stamina is depleted.
- **OnRestFinished** — broadcast when the actor stops resting.
- **OnStartRest** — broadcast when the actor starts resting.